Session 2: Variance and Hypothesis

Combining Data, Descriptive Statistics, and Understanding Uncertainty

Karry Jiao
dj398@cam.ac.uk
KJ Github
Oct. 2026

Table of contents

1. Introduction
2. Lecture review
3. Combining and subsetting data
4. Quick summary statistics
5. Descriptive tables
6. Variance and standard deviation in R
7. Summaries by group
8. Applying uncertainty concepts
9. Loops and map

Introduction

What this session is about

By the end, you should be able to:

- combine datasets by adding rows or matching keys;
- select observations and variables for analysis;
- read and create summary statistics tables;
- calculate variance, standard deviation and summaries by group;
- calculate and interpret standard errors, confidence intervals and p-values; and
- recognize how loops and `map()` repeat a familiar calculation.

Examples use World Happiness Report data. Select one year for descriptive comparisons across countries.

Open your existing RStudio project and create `exercises-session2.R`.

Lecture Review

Lecture Review — Two kinds of variation

Across individuals

How different are people's outcomes within a sample?

Standard deviation (SD) describes this variation.

$$SD = s = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}}$$

Across samples

How much would our estimate change if we drew another sample?

Standard error (SE) describes variation in an estimate across repeated samples.

For the sample mean:

$$\widehat{SE}(\bar{x}) = \frac{s}{\sqrt{n}}$$

Example: Individuals' waiting times vary. The estimated average waiting time also varies across repeated samples.

Lecture Review — Variance and standard deviation

Population and sample variance

Population variance:

$$\sigma^2 = \frac{\sum_{i=1}^{N}(x_i - \mu)^2}{N}$$

Sample variance:

$$s^2 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}$$

Larger SD means greater variation across individuals.

Sample standard deviation

$$s = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}}$$

- x_i: an individual observation.
- μ, $\bar{x}$: population and sample means.
- N, n: population and sample sizes.
- Variance uses **squared units**.
- SD uses the **original units**.

Lecture Review — Sampling distributions

Repeated sampling

- Draw samples of the same size from one population.
- Calculate an estimate from each sample.
- The distribution of these estimates is a **sampling distribution**.

Here, the estimate is the sample mean, $\bar{x}$. Medians, proportions and differences in means also have sampling distributions and standard errors.

Lecture Review — Confidence intervals

A range around an estimate

Using a normal approximation:

$$95\%\ \text{CI} = \text{estimate} \pm 1.96 \times SE$$

School meals increase attendance by an estimated **3 percentage points**, with $SE = 1.54$:

$$3 \pm 1.96(1.54) = [-0.02,\ 6.02]$$

Interpretation

- The interval includes zero and potentially substantial positive effects.
- Across repeated samples, approximately **95% of intervals constructed this way** would contain the true effect.
- The endpoints are **not guaranteed minimum and maximum effects**.

Lecture Review — Bias and precision

Concept	Meaning	Example
Bias	Systematic deviation from the true population value	Surveying only visitors during busy hours
Precision	How closely estimates cluster across repeated samples	A large random sample produces a smaller SE

- An estimate can be **precise yet biased**.
- A small random sample can produce an **unbiased but imprecise** estimate.
- Larger samples reduce random sampling uncertainty, but do **not automatically correct selection bias or coverage errors**.

Discussion: Would data covering every walk-in visitor represent all visitors if they excluded people with appointments?

Lecture Review — Hypothesis tests, z-statistics, p-value

Null and alternative hypotheses

For the true average effect, Δ:

$$H_0 : \Delta = 0 \qquad H_1 : \Delta \neq 0$$

Distance from the null value in standard errors:

$$z = \frac{\text{estimate} - \text{null value}}{SE}$$

For a two-sided standard normal test:

$$p = 2[1 - \Phi(|z|)]$$

Lecture example

$$z = \frac{3 - 0}{1.54} \approx 1.95 \qquad p \approx 0.051$$

Under the no-effect model, results at least this far from zero in either direction occur about **5.1%** of the time.

$\Phi(z)$ is the standard normal probability to the left of z.

Standard normal Z-table — University of Arizona

Remember: The p-value does not tell us the probability that the null hypothesis is true.

Combining and subsetting data

Packages and data for this session

Recommend use the same project and data folders as in Session 1. If not, then don't worry, just find your datasets and read in using the common way of loading data.

```
# Install once if needed
install.packages(c("tidyverse", "here", "janitor", "modelsummary"))
```

```
library(tidyverse)
library(here)
library(janitor)
library(modelsummary)
```

Data files: `WHR2015.csv`, `WHR2016.csv`, `WHR2017.csv` and `regions.RDS` in `DataWork/DataSets/Raw/Un WHR`.

🔄 RECAP: Loading a Dataset in R

Exercise: Loading data using the `here` package

Load the three WHR datasets from the folder:

📂 `DataWork/DataSets/Raw/Un WHR`

Name each dataset using the format `whrYY`, where `YY` represents the last two digits of the year. For example, `WHR2015.csv` should be named `whr15`.

🔄 RECAP: Loading a Dataset in R

Solution:

```
whr15 ← read_csv(here("DataWork", "DataSets", "Raw", "Un WHR", "WHR2015.csv"))
whr16 ← read_csv(here("DataWork", "DataSets", "Raw", "Un WHR", "WHR2016.csv"))
whr17 ← read_csv(here("DataWork", "DataSets", "Raw", "Un WHR", "WHR2017.csv"))
```

The `janitor` Package: Clean Column Names Easily

- Sometimes datasets have **messy column names** (e.g., inconsistent capitalization, spaces, or special characters).
- The `clean_names()` function (from the `janitor` package) automatically **fixes** variable names into a **consistent, lowercase, and snake_case format**.

> **Tip**: You can pipe `clean_names()` right after loading a dataset!

```
whr15 ← whr15 %>%
  clean_names()
whr16 ← whr16 %>%
  clean_names()
whr17 ← whr17 %>%
  clean_names()
```

If we want to to rename our variable manually (not recommended), we could use:

```
whr15 ← whr15 %>%
  rename(
    var_newname = var_oldname
  )
```

Exploring a Dataset: First Steps in Data Wrangling

Why is this important?

Before analyzing or wrangling data, we need to **understand** what we're working with.
Below are the **essential functions** we discussed during lab session1 to explore datasets in R.

Exploring Data with Base R

These functions help you **inspect your dataset**:

```
View(data)      # Open the dataset in a viewer
class(data)     # Object type (data frame, list, etc.)
dim(data)       # Number of rows & columns
names(data)     # Variable names
str(data)       # Structure of the dataset
summary(data)   # Summary statistics
head(data)      # First few observations
tail(data)      # Last few observations
```

Exploring Data with the Tidyverse

If you're using the tidyverse, this function is super useful:

```
glimpse(data)   # Compact overview of dataset
```

✔ More readable than str() ✔ Displays variable types and first few values

Dimensions of your data

Let's see how many columns and observations the dataset has:

- **Dimensions of your data** (Rows and Columns):

```
dim(whr15)
```

```
## [1] 158  12
```

- **The number of distinct values of a particular variable**:

```
n_distinct(DATASET$variable, na.rm = TRUE)
```

Dimensions of your data

Let's see how many columns and observations the dataset has:

- **Dimensions of your data** (Rows and Columns):

```
dim(whr15)
```

```
## [1] 158  12
```

- **The number of distinct values of a particular variable**:

```
n_distinct(DATASET$variable, na.rm = TRUE)
```

The `$` sign is a subsetting operator. In R, we have three subsetting operators (`[[`, `[`, and `$`.). It is often used to access variables in a dataframe.

The `n_distinct` function allows us to count the number of unique values of a variable length of a vector. We included `na.rm = TRUE`, so we don't count missing values.

Keys and matching relationships

A **key** identifies an observation. In an annual WHR file, it is `country`. After appending years, it is `country` and `year` together.

Relationship	Meaning	Example
1:1	Key is unique in both tables	One year's happiness scores and country information
Many:1	Repeated keys on the left, unique keys on the right	Country-year observations and a country-region lookup

Left/right/inner specify which observations to retain. **1:1/many:1** specify how keys match.

```
whr15 %>% count(country) %>% filter(n > 1)
```

```
## # A tibble: 0 × 2
## # ℹ 2 variables: country <chr>, n <int>
```

An empty result means no duplicated country keys. Missing keys still require a separate check.

Filtering and sorting dplyr::filter

- The `filter` function is used to subset rows in a dataset.

```
whr15 %>% filter(region == "Western Europe")
```

```
## # A tibble: 21 × 12
##    country     region         happiness_rank happiness_score standard_error
##    <chr>       <chr>                   <dbl>           <dbl>          <dbl>
##  1 Switzerland Western Europe              1            7.59         0.0341
##  2 Iceland     Western Europe              2            7.56         0.0488
##  3 Denmark     Western Europe              3            7.53         0.0333
##  4 Norway      Western Europe              4            7.52         0.0388
##  5 Finland     Western Europe              6            7.41         0.0314
##  6 Netherlands Western Europe              7            7.38         0.0280
##  7 Sweden      Western Europe              8            7.36         0.0316
##  8 Austria     Western Europe             13            7.2          0.0375
##  9 Luxembourg  Western Europe             17            6.95         0.0350
## 10 Ireland     Western Europe             18            6.94         0.0368
## # ℹ 11 more rows
## # ℹ 7 more variables: economy_gdp_per_capita <dbl>, family <dbl>,
## #   health_life_expectancy <dbl>, freedom <dbl>,
## #   trust_government_corruption <dbl>, generosity <dbl>,
## #   dystopia_residual <dbl>
```

Filtering and sorting dplyr::filter

Exercise: Use `filter()` to extract rows in these regions: (1) Eastern Asia and (2) North America. Hint: use the **or** operator (`|`):

Filtering and sorting dplyr::filter

Exercise: Use `filter()` to extract rows in these regions: (1) Eastern Asia and (2) North America. Hint: use the **or** operator (`|`):

Solution:

```
whr15 %>%
  filter(region == "Eastern Asia" | region == "North America")
```

A more elegant approach would be to use the `%in%` operator (equivalent to `inlist()` in Stata):

```
whr15 %>%
  filter(region %in% c("Eastern Asia", "North America"))
```

Filtering and sorting missing cases

If you want to remove (or identify) missing cases for a specific variable, you can use `is.na()`.

- This function returns a value of true and false for each value in a data set.
- If the value is *NA* the `is.na()` function returns TRUE, otherwise, it returns FALSE.
- In this way, we can check NA values that can be used for other functions.
- We can also negate the function using `!is.na()` which indicates that we want to return those observations with no missing values in a variable.

The function syntax in a pipeline is as follows:

```
DATA %>%
  filter(
    is.na(VAR)
  )
```

What are we returning here?

Filtering and sorting missing cases

If you want to remove (or identify) missing cases for a specific variable, you can use `is.na()`.

- This function returns a value of true and false for each value in a data set.
- If the value is *NA* the `is.na()` function returns TRUE, otherwise, it returns FALSE.
- In this way, we can check NA values that can be used for other functions.
- We can also negate the function using `!is.na()` which indicates that we want to return those observations with no missing values in a variable.

The function syntax in a pipeline is as follows:

```
DATA %>%
  filter(
    is.na(VAR)
  )
```

What are we returning here?

The observations with missing values for the variable VAR.

Filtering and sorting missing cases

Let's try filtering the `whr15` data. Let's keep those observations that have information per region, i.e., no missing values.

```
whr15 %>%
  filter(!is.na(region)) %>%
  head(5)
```

```
## # A tibble: 5 × 12
##   country     region         happiness_rank happiness_score standard_error
##   <chr>       <chr>                   <dbl>           <dbl>          <dbl>
## 1 Switzerland Western Europe              1            7.59         0.0341
## 2 Iceland     Western Europe              2            7.56         0.0488
## 3 Denmark     Western Europe              3            7.53         0.0333
## 4 Norway      Western Europe              4            7.52         0.0388
## 5 Canada      North America               5            7.43         0.0355
## # ℹ 7 more variables: economy_gdp_per_capita <dbl>, family <dbl>,
## #   health_life_expectancy <dbl>, freedom <dbl>,
## #   trust_government_corruption <dbl>, generosity <dbl>,
## #   dystopia_residual <dbl>
```

> Notice that we are negating the function, i.e., `!is.na()`
> In case we want to keep the observations that contains missing information we will only use `is.na()`.

Filtering

We can use the `%in%` operator to test if values belong to a vector. To **filter out** values, we use the **negation** `!%in%`.

For instance, if we want to exclude the regions "Eastern Asia" and "North America":

```
whr15 %>%
  filter(!region %in% c("Eastern Asia", "North America")) %>%
  head(5)
```

```
## # A tibble: 5 × 12
##   country     region         happiness_rank happiness_score standard_error
##   <chr>       <chr>                   <dbl>           <dbl>          <dbl>
## 1 Switzerland Western Europe              1            7.59         0.0341
## 2 Iceland     Western Europe              2            7.56         0.0488
## 3 Denmark     Western Europe              3            7.53         0.0333
## 4 Norway      Western Europe              4            7.52         0.0388
## 5 Finland     Western Europe              6            7.41         0.0314
## # ℹ 7 more variables: economy_gdp_per_capita <dbl>, family <dbl>,
## #   health_life_expectancy <dbl>, freedom <dbl>,
## #   trust_government_corruption <dbl>, generosity <dbl>,
## #   dystopia_residual <dbl>
```

This filters the data to show only rows where `region` is **not** in the specified list.

Creating variables

Exercise: Create a variable called `year` that equals to the year of each dataframe using the `mutate()`. Remember to assign it to the same dataframe.

Creating variables

Exercise: Create a variable called `year` that equals to the year of each dataframe using the `mutate()`. Remember to assign it to the same dataframe.

Solution:

```
whr15 ← whr15 %>%
  mutate(year = 2015)

whr16 ← whr16 %>%
  mutate(year = 2016)

whr17 ← whr17 %>%
  mutate(year = 2017)
```

Appending dataframes

Now that we can identify the observations, we can combine the data set. Here are two functions to append objects by row

```
rbind(df1, df2, df3) # The base R function

bind_rows(df1, df2, df3) # The dplyr function, making some improvements to base R
```

Exercise: Append data sets. Use the function `bind_rows` to append the three WHR datasets:

Appending dataframes

Now that we can identify the observations, we can combine the data set. Here are two functions to append objects by row

```
rbind(df1, df2, df3) # The base R function

bind_rows(df1, df2, df3) # The dplyr function, making some improvements to base R
```

Exercise: Append data sets. Use the function `bind_rows` to append the three WHR datasets:

Solution:

```
bind_rows(whr15, whr16, whr17)
```

Appending dataframes

Now that we can identify the observations, we can combine the data set. Here are two functions to append objects by row

```
rbind(df1, df2, df3) # The base R function

bind_rows(df1, df2, df3) # The dplyr function, making some improvements to base R
```

Exercise: Append data sets. Use the function `bind_rows` to append the three WHR datasets:

Solution:

```
bind_rows(whr15, whr16, whr17)
```

Notes

- One of the problems with binding rows like this is that, sometimes, some columns are not *fully* compatible.

Appending and merging data sets

Exercise: Fixing our variables and appending the data frames correctly.

Exercise

- Load the R data set `regions.RDS` from `DataWork/DataSets/Raw/Un WHR` using the `read_rds` function.

Appending and merging data sets

Exercise: Fixing our variables and appending the data frames correctly.

Exercise

- Load the R data set `regions.RDS` from `DataWork/DataSets/Raw/Un WHR` using the `read_rds` function.

Solution:

```
regions ← read_rds(here("DataWork", "DataSets", "Raw", "Un WHR", "regions.RDS"))
```

Appending and merging data sets

We can use the `dplyr::left_join()` function to merge two dataframes. The function syntax is: `left_join(a_df, another_df, by = c("id_col1"))`.

> A left join takes all the values from the first table, and looks for matches in the second table. If it finds a match, it adds the data from the second table; if not, it adds missing values. It is the equivalent of `merge, keep(master matched)` in Stata.

Left, right, inner and full joins

```
a ← tibble(id = c(1, 2), score = c(10, 20))
b ← tibble(id = c(2, 3), region = c("East", "West"))
```

Command	IDs retained	What happens to unmatched observations?
`left_join(a, b, by = "id")`	1, 2	Missing region for ID 1
`right_join(a, b, by = "id")`	2, 3	Missing score for ID 3
`inner_join(a, b, by = "id")`	2	Unmatched observations excluded
`full_join(a, b, by = "id")`	1, 2, 3	Missing values where a match is absent

Exercise: Run each command and explain why the rows differ.

Appending and merging data sets

Exercise: Join the dataframes: `regions` and `whr17`.

Appending and merging data sets

Exercise: Join the dataframes: `regions` and `whr17`.

Solution:

```
whr17 ← whr17 %>%
  left_join(regions, by = "country") %>%
  select(country, region, everything())
```

Notes:

Look at the `everything()` function. It takes all the variables from the dataframe and put them after country and region. In this way, select can be use to **order** columns!

Appending and merging data sets

Exercise: Check if there is any other countries in `whr17` without region info:

- Only use pipes %>%
- And `filter()`
- Do not assign it to an object.

Appending and merging data sets

Exercise: Check if there is any other countries in `whr17` without region info:

- Only use pipes %>%
- And `filter()`
- Do not assign it to an object.

Solution:

```
whr17 %>%
  filter(is.na(region))
```

```
## # A tibble: 2 × 14
##   country         region happiness_rank happiness_score whisker_high whisker_low
##   <chr>           <chr>           <dbl>           <dbl>        <dbl>       <dbl>
## 1 Taiwan Provinc… <NA>               33            6.42         6.49        6.35
## 2 Hong Kong S.A.… <NA>               71            5.47         5.55        5.39
## # ℹ 8 more variables: economy_gdp_per_capita <dbl>, family <dbl>,
## #   health_life_expectancy <dbl>, freedom <dbl>, generosity <dbl>,
## #   trust_government_corruption <dbl>, dystopia_residual <dbl>, year <dbl>
```

So we ended up with two regions with NAs

This is due to the name of the regions. The regions dataset doesn't have "Taiwan Province of China" nor "Hong Kong S.A.R., China" but "Taiwan" and "Hong Kong."

How do you think we should solve this?

So we ended up with two regions with NAs

This is due to the name of the regions. The regions dataset doesn't have "Taiwan Province of China" nor "Hong Kong S.A.R., China" but "Taiwan" and "Hong Kong."

How do you think we should solve this?

- My approach would be to:

1. fix the names of these regions in the `whr17` dataset (a data cleaning task) and;
2. merge (left_join) it with the regions dataset.

Correcting country names before a join

Correct the names **before joining**. Reload the original file to avoid joining twice:

```
whr17 ← read_csv(here("DataWork", "DataSets", "Raw", "Un WHR", "WHR2017.csv")) %>%
  clean_names() %>%
  mutate(year = 2017,
         country = recode(country,
           "Taiwan Province of China" = "Taiwan",
           "Hong Kong S.A.R., China" = "Hong Kong")) %>%
  left_join(regions, by = "country")
```

Appending and merging data sets

Finally, let's keep those relevant variables first and bind those rows.

Exercise: Bind all rows and create a panel called: `whr_panel`.

- Select the variables: `country`, `region`, `year`, `happiness_rank`, `happiness_score`, `economy_gdp_per_capita`, `health_life_expectancy`, `freedom` for each df, i.e., `whr15`, `whr16`, `whr17`.
- Use `rbind()`

Appending and merging data sets

Finally, let's keep those relevant variables first and bind those rows.

Exercise: Bind all rows and create a panel called: `whr_panel`.

- Select the variables: `country`, `region`, `year`, `happiness_rank`, `happiness_score`, `economy_gdp_per_capita`, `health_life_expectancy`, `freedom` for each df, i.e., `whr15`, `whr16`, `whr17`.
- Use `rbind()`

Solution:

```
vars_to_keep ← c("country", "region", "year", "happiness_rank",
                 "happiness_score", "economy_gdp_per_capita",
                 "health_life_expectancy", "freedom")

whr15 ← select(whr15, all_of(vars_to_keep))
whr16 ← select(whr16, all_of(vars_to_keep))
whr17 ← select(whr17, all_of(vars_to_keep))

whr_panel ← rbind(whr15, whr16, whr17)    # or bind_rows
```

Checking the combined data

```
whr_panel %>% count(country, year) %>% filter(n > 1)
```

```
## # A tibble: 0 × 3
## # ℹ 3 variables: country <chr>, year <dbl>, n <int>
```

```
whr_panel %>% filter(is.na(region))
```

```
## # A tibble: 0 × 8
## # ℹ 8 variables: country <chr>, region <chr>, year <dbl>, happiness_rank <dbl>,
## #   happiness_score <dbl>, economy_gdp_per_capita <dbl>,
## #   health_life_expectancy <dbl>, freedom <dbl>
```

```
nrow(whr_panel)
```

```
## [1] 470
```

For a many:1 region join, verify that the lookup's country key is unique. Duplicate lookup keys can multiply rows.

```
regions %>% count(country) %>% filter(n > 1)
anti_join(whr17, regions, by = "country") # unmatched country keys
```

An unmatched key differs from a matched key whose region value is missing.

Subsetting for the descriptive analysis

Use **one year** so each row represents one country.

dplyr

```
whr ← whr_panel %>%
  filter(year == 2015) %>%
  select(country, region, happiness_score,
         health_life_expectancy, freedom)
```

Base R

```
whr ← subset(
  whr_panel,
  year == 2015,
  select = c(country, region,
    happiness_score,
    health_life_expectancy, freedom)
)
```

Quick summary statistics

Exploring a dataset

`summary(x, digits)`

Produces a compact summary of an R object. Its arguments are:

- **x:** the object you want to summarize, usually a vector or data frame
- *digits:* the number of significant digits used for display

Exploring a dataset

`summary(x, digits)`

Produces a compact summary of an R object. Its arguments are:

- **x:** the object you want to summarize, usually a vector or data frame
- *digits:* the number of significant digits used for display

Exercise ⌨

Use the `summary()` function to describe the `whr` data frame.

Exploring a dataset

```
summary(whr)
```

```
##    country             region          happiness_score health_life_expectancy
##  Length:158         Length:158         Min.   :2.839   Min.   :0.0000
##  Class :character   Class :character   1st Qu.:4.526   1st Qu.:0.4392
##  Mode  :character   Mode  :character   Median :5.232   Median :0.6967
##                                        Mean   :5.376   Mean   :0.6303
##                                        3rd Qu.:6.244   3rd Qu.:0.8110
##                                        Max.   :7.587   Max.   :1.0252
##     freedom
##  Min.   :0.0000
##  1st Qu.:0.3283
##  Median :0.4355
##  Mean   :0.4286
##  3rd Qu.:0.5491
##  Max.   :0.6697
```

Reading a numeric summary

Label	Meaning
Min. / Max.	Smallest / largest observed value
1st Qu. / 3rd Qu.	25th / 75th percentile
Median	50th percentile
Mean	Arithmetic average
NA's	Missing observations, when present

Question: If the mean exceeds the median, what might that suggest about the distribution?

`summary()` does not normally include SD or SE for a numeric variable.

Summarizing continuous variables

- `summary()` can also be used with a single variable.
- When used with continuous variables, it works similarly to `summarize` in Stata.
- When used with factors, it reports category counts. Character columns may instead show length, class and mode.

Summarizing continuous variables

Exercise ⌨

Use the `summary()` function to display summary statistics for a continuous variable in the `whr` data frame.

Summarizing continuous variables

Exercise ⌨

Use the `summary()` function to display summary statistics for a continuous variable in the `whr` data frame.

```
summary(whr$happiness_score)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   2.839   4.526   5.232   5.376   6.244   7.587
```

Summarizing categorical variables

`tabyl(x, ...)`

Equivalent to `tabulate` in Stata, creates a frequency table. Its main arguments are vectors to be tabulated.

- **x:** the object you want to summarize, usually a vector or data frame
- ... additional options as show_na, or show_missing_levels.

Summarizing categorical variables

`tabyl(x, ...)`

Equivalent to `tabulate` in Stata, creates a frequency table. Its main arguments are vectors to be tabulated.

- **x:** the object you want to summarize, usually a vector or data frame
- ... additional options as show_na, or show_missing_levels.

Exercise ⌨

Use the `tabyl()` function to display frequency tables for:

1. The variable `region` in the `whr` data frame
2. The variables `region` and `year` in the `whr_panel` data frame, simultaneously

Summarizing categorical variables

One way tabulation

```
whr %>%
  tabyl(region)
```

```
##                            region  n    percent
##         Australia and New Zealand  2 0.01265823
##        Central and Eastern Europe 29 0.18354430
##                       Eastern Asia  6 0.03797468
##       Latin America and Caribbean 22 0.13924051
##  Middle East and Northern Africa 20 0.12658228
##                      North America  2 0.01265823
##                  Southeastern Asia  9 0.05696203
##                      Southern Asia  7 0.04430380
##                 Sub-Saharan Africa 40 0.25316456
##                     Western Europe 21 0.13291139
```

Summarizing categorical variables

Two way tabulation

```
whr_panel %>% tabyl(region, year)
```

```
##                           region 2015 2016 2017
##          Australia and New Zealand    2    2    2
##         Central and Eastern Europe   29   29   29
##                       Eastern Asia    6    6    6
##        Latin America and Caribbean   22   24   22
##  Middle East and Northern Africa   20   19   19
##                      North America    2    2    2
##                  Southeastern Asia    9    9    8
##                      Southern Asia    7    7    7
##                 Sub-Saharan Africa   40   38   39
##                     Western Europe   21   21   21
```

Each cell counts country-year observations in that region and year.

Question: Why can counts differ between years? Does a larger count mean a happier region?

Descriptives tables

Exploring datasets with *modelsummary*

The package *modelsummary* contains a family of functions called `datasummary` which can be used to create different types of summary statistics tables. These include:

- `datasummary_skim`, to create descriptive statistics tables
- `datasummary_crosstab`, to create a twoway tabulation
- `datasummary`, to create customized descriptive statistics tables

Exploring datasets with *modelsummary*

`datasummary_skim(data, output, ....)`

- **data:** the data set to be summarized, the only required argument
- **output:** the type of output desired
- ...: additional options allow for formatting customization, such as including notes and titles

```
datasummary_skim(
  data,
  type = "numeric",
  output = "kableExtra",
  title = NULL,
  notes = NULL,
  ...
)
```

Exploring datasets with *modelsummary*

Exercise ⌨

Use `datasummary_skim()` to create a descriptive statistics table for the `whr` data.

Exploring datasets with *modelsummary*

```
datasummary_skim(whr, type = "numeric", output = "kableExtra")
```

	Unique	Missing Pct.	Mean	SD	Min	Median	Max
happiness_score	157	0	5.4	1.1	2.8	5.2	7.6
health_life_expectancy	157	0	0.6	0.2	0.0	0.7	1.0
freedom	158	0	0.4	0.2	0.0	0.4	0.7

Reading a descriptive table

- **Unique:** number of distinct values, not number of observations.
- **Missing Pct.:** percentage of missing observations.
- **Mean / Median:** measures of centre.
- **SD:** variation across observations, in the variable's original units.
- **Min / Max:** observed extremes.

For this table, observations are **countries in 2015**. The average gives equal weight to each observed country.

SD describes spread across countries. It is not the standard error of the mean.

Exploring datasets with *modelsummary*

- *modelsummary* summarizes all variables by default.
- To summarize only categorical variables, use the argument `type`

```
datasummary_skim(whr %>% select(region), type = "categorical", output = "tinytable")
datasummary_skim(whr, type = "numeric", output = "kableExtra")
```

region	N	%
Australia and New Zealand	2	1.3
Central and Eastern Europe	29	18.4
Eastern Asia	6	3.8
Latin America and Caribbean	22	13.9
Middle East and Northern Africa	20	12.7
North America	2	1.3
Southeastern Asia	9	5.7
Southern Asia	7	4.4
Sub-Saharan Africa	40	25.3

Exploring datasets with *modelsummary*

You can also customize the variables and statistics to include using a **formula** with the `datasummary()` function.

`datasummary(formula, data, output, ...)`

- **formula:** a two-sided formula to describe the table: rows ~ columns
- **data:** the data set to be summarized
- **output:** the type of output desired
- ...: additional options allow for formatting customization

```
datasummary(
  var1 + var2 + var3 ~ stat1 + stat2 + stat3 + stat4,
  data = data
)
```

Exploring datasets with *modelsummary*

Exercise ⌨

Create a table showing the number of observations, mean, standard deviation, minimum, maximum and median value for happiness scores, health life expectancy and freedom in the `whr` data.

```
datasummary(
  happiness_score + health_life_expectancy + freedom ~ N + Mean + SD + Median + Min + Max,
  data = whr, output = "kableExtra"
)
```

> **Tip:** some of the allowed statistics are N, Mean, SD, Min, Max, Median, P0, P25, P50, P75, P100, Histogram

Exploring datasets with *modelsummary*

```
datasummary(
  happiness_score + health_life_expectancy + freedom ~ N + Mean + SD + Median + Min + Max,
  data = whr, output = "kableExtra"
)
```

	N	Mean	SD	Median	Min	Max
happiness_score	158	5.38	1.15	5.23	2.84	7.59
health_life_expectancy	158	0.63	0.25	0.70	0.00	1.03
freedom	158	0.43	0.15	0.44	0.00	0.67

Variance and standard deviation in R

Variance and standard deviation: Base R

```
x ← whr$happiness_score
var(x, na.rm = TRUE)  # sample variance
```

```
## [1] 1.311048
```

```
sd(x, na.rm = TRUE)   # sample standard deviation
```

```
## [1] 1.14501
```

```
sqrt(var(x, na.rm = TRUE)) # same as sd()
```

```
## [1] 1.14501
```

- `var()` and `sd()` use the **sample denominator, n - 1**.
- `na.rm = TRUE` excludes missing values from the calculation.
- Variance uses squared units. SD uses the original units.
- With fewer than two nonmissing values, sample variance and SD are undefined (`NA`).

Variance and standard deviation: dplyr

```
whr %>%
  summarise(
    n_observed = sum(!is.na(happiness_score)),
    mean_happiness = mean(happiness_score, na.rm = TRUE),
    median_happiness = median(happiness_score, na.rm = TRUE),
    variance_happiness = var(happiness_score, na.rm = TRUE),
    sd_happiness = sd(happiness_score, na.rm = TRUE)
  )
```

```
## # A tibble: 1 × 5
##   n_observed mean_happiness median_happiness variance_happiness sd_happiness
##        <int>          <dbl>            <dbl>              <dbl>        <dbl>
## 1        158           5.38             5.23               1.31         1.15
```

`dplyr` organizes the calculation. The underlying functions are the same Base R `mean()`, `var()` and `sd()`.

Exercise: Calculate variance and SD for `freedom`. Explain the units of each.

Summaries by group

Aggregating observations

- If you want to show aggregated statistics, the function `summarise` is a powerful tool.
- It is similar to `datasummary` in that it calculates a series of statistics for a data frame.
- However, it does not have pre-defined statistics, so it requires more manual input.
- On the other hand, its output is a regular data frame, so it is also useful to create constructed data sets.
- Its Stata equivalent would be `collapse`

Aggregating observations

`summarise(.data, ...,)`

- **data**: the data frame to be summarized
- **...**: Name-value pairs of summary functions. The name will be the name of the variable in the result.

Aggregating observations

`summarise(.data, ... ,)`

- **data**: the data frame to be summarized
- **...**: Name-value pairs of summary functions. The name will be the name of the variable in the result.

The "name-value" pairs mentioned under `...` look like this: `new_variable = function(existing_variable)`, where possible functions include:

- Center: `mean()`, `median()`
- Spread: `sd()`, `IQR()`, `mad()`
- Range: `min()`, `max()`, `quantile()`
- Count: `n()`, `n_distinct()`

Aggregating observations by region

Adapt the source exercise to WHR countries:

```
region_stats ← whr %>%
  group_by(region) %>%
  summarise(
    n_observed = sum(!is.na(happiness_score)),
    mean_happiness = mean(happiness_score, na.rm = TRUE),
    median_happiness = median(happiness_score, na.rm = TRUE),
    sd_happiness = sd(happiness_score, na.rm = TRUE),
    .groups = "drop"
  )
knitr::kable(region_stats, digits = 2)
```

region	n_observed	mean_happiness	median_happiness	sd_happiness
Australia and New Zealand	2	7.28	7.28	0.00
Central and Eastern Europe	29	5.33	5.29	0.57
Eastern Asia	6	5.63	5.73	0.55
Latin America and Caribbean	22	6.14	6.15	0.73
Middle East and Northern Africa	20	5.41	5.26	1.10

Applying uncertainty concepts

Standard error and confidence interval in R

Illustrative independent observations of waiting times, in minutes:

```
waiting ← c(12, 18, 20, 22, 24, 25, 28, 30, 35, 40)
x_mean ← mean(waiting)
x_sd ← sd(waiting)
x_n ← length(waiting)
x_se ← x_sd / sqrt(x_n)
c(mean = x_mean, SD = x_sd, SE = x_se)
```

```
##      mean        SD        SE
## 25.400000  8.235425  2.604270
```

```
c(lower = x_mean - 1.96 * x_se,
  upper = x_mean + 1.96 * x_se)
```

```
##    lower    upper
## 20.29563 30.50437
```

This uses the lecture's **normal approximation**. For a small sample with unknown population SD, a t-based interval is usually more appropriate when its assumptions hold.

From the lecture's effect estimate to a p-value

```r
effect ← 3   # percentage points
se_effect ← 1.54
z ← (effect - 0) / se_effect
p_value ← 2 * pnorm(-abs(z))
c(z = z, p_value = p_value)
```

```
##          z    p_value
## 1.94805195 0.05140875
```

```r
c(lower = effect - 1.96 * se_effect,
  upper = effect + 1.96 * se_effect)
```

```
##   lower   upper
## -0.0184  6.0184
```

`pnorm()` gives a cumulative probability under the standard normal distribution. The calculation counts both tails.

Standard normal Z-table — University of Arizona

Interpretation: Under the no-effect model, results at least this extreme occur about 5.1% of the time.

Loops and map

Vectorization, loops, and `map()`

Use the simplest appropriate tool:

Tool	Best suited to	Typical output
Vectorized operation	an operation already defined for a whole vector	vector
`for` loop	explicit control over steps or side effects	whatever you build or change
`purrr::map()`	applying one function consistently to each element	list

```
x ← c(1.2, 2.5, 9.1, 5.8)

round(x)          # vectorized: simplest here
map(x, round)     # iteration with a list result
```

`map()` is not automatically parallel and is not always faster; its main benefit is a clear, consistent functional pattern.

Map

- R, however, has a set of functions that allows users to iterate over an object **without writing an explicit loop**
- In this training we'll introduce `map()`. It is a function part of `purrr`, a package that contains tools for functional programming
- Also, in case you have not noticed yet: **R is vectorized!** this means that many operations are applied element-wise by default so you don't have to code loops to apply them to each element of a vector or dataframe

Map

- To use `map()`, you need to load the package `purrr`
- The basic syntax of `map()` is:

`map(X, function, ...)`: applies `function` to each of the elements of `X`. If `X` is a dataframe then `function` is applied column-wise while if it's a vector or a list it is applied item-wise. The output of `map()` is always a list with the results.

- **X:** a data frame, list or vector the function will be applied to
- **function:** the name of the function you want to apply to each of the elements of `X`

Map

```r
# Round the values of the following vector
x ← c(1.2, 2.5, 9.1, 5.8)
map(x, round)    # Returns a list; same as x %>% map(round)
round(x)         # since R is vectorized, this also works
```

A loop and `map()` compared

Explicit loop

```
df ← whr %>% select(where(is.numeric))
means_loop ← numeric(ncol(df))

for (i in seq_along(df)) {
  means_loop[i] ← mean(df[[i]], na.rm = TRUE)
}
```

Functional iteration

```
means_map ← map(df, mean, na.rm = TRUE)

# Return a numeric vector
means_dbl ← map_dbl(df, mean, na.rm = TRUE)
```

Preallocating the loop output is preferable to repeatedly using `append()`.

Key takeaways

- Appending adds observations. Joins match observations using keys.
- Check missing keys, duplicate keys and unmatched observations.
- Read a table's units, sample size and missing values before interpreting its statistics.
- `var()` and `sd()` describe spread. SE describes uncertainty in an estimate.
- Compare groups using both their averages and their variation.
- Loops and `map()` repeat calculations you already understand.

Credit to DIME Analytics, World Bank